Journal Assignment – 3

Q:- 1. Setup Your Database Connection: 1. Import Module, 2. Connect to Database, 3. Create a Cursor.

- **Input:-**
- **step1_setup_connection.py:-**

```
# 1. Import Module
import sqlite3

# 2. Connect to Database
connection = sqlite3.connect('store.db')
print("Successfully connected to 'store.db'.")

# 3. Create a Cursor
cursor = connection.cursor()
print("Cursor created successfully to execute SQL statements.")

# Close connection
connection.close()
print("Connection closed.")
```

- **Output:-**

```
Successfully connected to 'store.db'.
Cursor created successfully to execute SQL statements.
Connection closed
```

Q:- 2. Create Your First Table (products): 1. Define SQL, 2. Execute SQL, 3. Save Changes, 4. Close Connection.

- **Input:-**
- **step2_create_table.py:-**

```
import sqlite3

connection = sqlite3.connect('store.db')
cursor = connection.cursor()

# 1. Define SQL
create_table_sql = '''
CREATE TABLE IF NOT EXISTS products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
    quantity INTEGER NOT NULL
);
'''

# 2. Execute SQL
cursor.execute(create_table_sql)
print("Table 'products' created successfully.")

# 3. Save Changes
connection.commit()
print("Changes saved (committed).")

# 4. Close Connection
connection.close()
print("Connection closed.")
```

- **Output:-**

```
Table 'products' created successfully.
Changes saved (committed).
Connection closed.
```

Q:- 3. Add New Products (Create Operation - INSERT): 1. Re-connect and Get Cursor, 2. Insert Single Product, 3. Insert Multiple Products, 4. Confirm Insertions, 5. Commit and Close.

- **Input:-**
- **step3_insert_products.py:-**

```
import sqlite3
# 1. Re-connect and Get Cursor
conn = sqlite3.connect('store.db')
cur = conn.cursor(
# 2. Insert Single Product
cur.execute('''
INSERT INTO products (product_name, price, quantity)
VALUES (?, ?, ?);
''', ('Wireless Mouse', 450.00, 25))
print("Single product inserted: Wireless Mouse.")
 # 3. Insert Multiple Products
products_list = [
    ('Mechanical Keyboard', 2200.00, 15),
    ('HD Webcam', 1800.00, 5),
    ('USB-C Hub', 1250.00, 4),
    ('Gaming Headset', 2800.00, 18)
]
cur.executemany('''
INSERT INTO products (product_name, price, quantity)
VALUES (?, ?, ?);
''', products_list)
print("Multiple products inserted successfully.")

# 4. Confirm Insertions
cur.execute("SELECT COUNT(*) FROM products;")
total_count = cur.fetchone()[0]
print(f"Total products now in table: {total_count}")
# 5. Commit and Close
conn.commit()
conn.close()
print("Transaction committed and connection closed.")
```

- **Output:-**

```
Single product inserted: Wireless Mouse.
Multiple products inserted successfully.
Total products now in table: 5
Transaction committed and connection closed.
```

Q:- 4. View Your Products (Read Operation - SELECT): 1. Print Products, 2. Select Specific Product, 3. Select Products with Low Quantity.

- **Input:-**
- **step4_view_products.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Print All Products
print("--- 1. All Products in Inventory ---")
cur.execute("SELECT * FROM products;")
for row in cur.fetchall():
    print(f"ID: {row[0]:2d} | Name: {row[1]:<20} | Price: Rs.{row[2]:7.2f} | Quantity: {row[3]}"
# 2. Select Specific Product by Name
print("\n--- 2. Select Specific Product ('Mechanical Keyboard') ---")
cur.execute("SELECT * FROM products WHERE product_name = ?;", ('Mechanical
Keyboard',))
prod = cur.fetchone()
print(f"Found: ID: {prod[0]} | Name: {prod[1]} | Price: Rs.{prod[2]} | Stock: {prod[3]}")

# 3. Select Products with Low Quantity (Quantity < 10)
print("\n--- 3. Products with Low Stock (Quantity < 10) ---")
cur.execute("SELECT product_name, quantity FROM products WHERE quantity < 10;")
for p in cur.fetchall():
    print(f"Low Stock Alert: {p[0]} (Only {p[1]} units left!)")

conn.close()
```

- **Output:-**

```
--- 1. All Products in Inventory ---
ID:  1 | Name: Wireless Mouse       | Price: Rs. 450.00 | Quantity: 25
ID:  2 | Name: Mechanical Keyboard  | Price: Rs.2200.00 | Quantity: 15
ID:  3 | Name: HD Webcam            | Price: Rs.1800.00 | Quantity: 5
ID:  4 | Name: USB-C Hub            | Price: Rs.1250.00 | Quantity: 4
ID:  5 | Name: Gaming Headset       | Price: Rs.2800.00 | Quantity: 18
--- 2. Select Specific Product ('Mechanical Keyboard') ---
Found: ID: 2 | Name: Mechanical Keyboard | Price: Rs.2200.0 | Stock: 15

--- 3. Products with Low Stock (Quantity < 10) ---
Low Stock Alert: HD Webcam (Only 5 units left!)
Low Stock Alert: USB-C Hub (Only 4 units left!)
```

Q:- 5. Update Product Information (Update Operation - UPDATE): 1. Update Quantity, 2. Update Price, 3. Confirm Updates.

- **Input:-**
- **step5_update_products.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Update Quantity of HD Webcam
cur.execute('''
UPDATE products
SET quantity = 30
WHERE product_name = 'HD Webcam';
''')
print("1. Quantity of 'HD Webcam' updated to 30.")

# 2. Update Price of Wireless Mouse
cur.execute('''
UPDATE products
SET price = 499.00
WHERE product_name = 'Wireless Mouse';
''')
print("2. Price of 'Wireless Mouse' updated to Rs. 499.00.")

conn.commit()

# 3. Confirm Updates
print("\n--- 3. Confirmation of Updated Products ---")
cur.execute("SELECT product_name, price, quantity FROM products WHERE product_name
IN ('HD Webcam', 'Wireless Mouse');")
for r in cur.fetchall():
    print(f"Product: {r[0]:<15} | New Price: Rs.{r[1]} | New Quantity: {r[2]}")

conn.close()
```

- **Output:-**

```
1. Quantity of 'HD Webcam' updated to 30.
2. Price of 'Wireless Mouse' updated to Rs. 499.00.
--- 3. Confirmation of Updated Products ---
Product: Wireless Mouse  | New Price: Rs.499.0 | New Quantity: 25
Product: HD Webcam       | New Price: Rs.1800.0 | New Quantity: 30
```

Q:- 6. Remove Products (Delete Operation - DELETE): 1. Delete a Product, 2. Confirm Deletion.

- **Input:-**
- **step6_delete_product.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Delete a Product (USB-C Hub)
cur.execute("DELETE FROM products WHERE product_name = 'USB-C Hub';")
conn.commit()
print("1. Product 'USB-C Hub' deleted from database.")

# 2. Confirm Deletion
print("\n--- 2. Inventory Check after Deletion ---")
cur.execute("SELECT product_id, product_name FROM products;")
remaining = cur.fetchall()
for p in remaining:
    print(f"ID: {p[0]} | Name: {p[1]}")

conn.close()
```

- **Output:-**

```
1. Product 'USB-C Hub' deleted from database.

--- 2. Inventory Check after Deletion ---
ID: 1 | Name: Wireless Mouse
ID: 2 | Name: Mechanical Keyboard
ID: 3 | Name: HD Webcam
ID: 5 | Name: Gaming Headset
```

Q:- 7. Understanding commit() and rollback(): 1. Demonstrate Commit, 2. Demonstrate Rollback.

- **Input:-**
- **step7_commit_and_rollback.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# 1. Demonstrate Commit
print("--- 1. Demonstrating commit() ---")
cur.execute("INSERT INTO products (product_name, price, quantity) VALUES ('Bluetooth
Speaker', 1499.0, 10);")
conn.commit()  # Changes are permanently saved
print("Product 'Bluetooth Speaker' inserted and committed.")

cur.execute("SELECT product_name FROM products WHERE product_name = 'Bluetooth
Speaker';")
print("Found after commit:", cur.fetchone()[0])

# 2. Demonstrate Rollback
print("\n--- 2. Demonstrating rollback() ---")
cur.execute("INSERT INTO products (product_name, price, quantity) VALUES ('Accidental
Dummy Product', 99999.0, 0);")
print("Inserted 'Accidental Dummy Product' (uncommitted).")

# Rollback changes before commit
conn.rollback()
print("Transaction rolled back (changes reverted)!")

cur.execute("SELECT product_name FROM products WHERE product_name = 'Accidental
Dummy Product';")
result = cur.fetchone()
print("Searching for uncommitted product after rollback:", result)
print("Observation: The uncommitted record was completely removed by rollback().")

conn.close()
```

- **Output:-**

```
--- 1. Demonstrating commit() ---
Product 'Bluetooth Speaker' inserted and committed.
Found after commit: Bluetooth Speaker
```

--- 2. Demonstrating rollback() ---
Inserted 'Accidental Dummy Product' (uncommitted).
Transaction rolled back (changes reverted)!
Searching for uncommitted product after rollback: None
Observation: The uncommitted record was completely removed by rollback().

Q:- 8. Basic Database Backup (Dump): 1. Dump Database, 2. Save to File.

- **Input:-**
- **step8_dump_backup.py:-**

```
import sqlite3

conn = sqlite3.connect('store.db')

# 1. Dump Database & 2. Save to File
backup_filename = 'store_backup.sql'
with open(backup_filename, 'w') as f:
    for line in conn.iterdump():
        f.write(f"{line}\n")

print(f"Database successfully dumped and backed up to '{backup_filename}'.")

# Preview the backup file content
print("\n--- Backup File Preview (First 5 lines) ---")
with open(backup_filename, 'r') as f:
    for i in range(5):
        print(f.readline().strip())

conn.close()
```

- **Output:-**

```
Database successfully dumped and backed up to 'store_backup.sql'.

--- Backup File Preview (First 5 lines) ---
BEGIN TRANSACTION;
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
```

Q:- 9. Export Products to CSV File.

- **Input:-**
- **step9_export_csv.py:-**

```
import sqlite3
```

```
import csv

conn = sqlite3.connect('store.db')
cur = conn.cursor()

# Retrieve all products
cur.execute("SELECT * FROM products;")
rows = cur.fetchall()

csv_file = 'products_exported.csv'
with open(csv_file, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Product_ID', 'Product_Name', 'Price', 'Quantity'])
    writer.writerows(rows)

print(f"Products exported successfully to '{csv_file}'.")

# Print CSV content
print("\n--- Exported CSV Contents ---")
with open(csv_file, 'r') as f:
    print(f.read())

conn.close()
```

- **Output:-**

```
Products exported successfully to 'products_exported.csv'.

--- Exported CSV Contents ---
Product_ID,Product_Name,Price,Quantity
1,Wireless Mouse,499.0,25
2,Mechanical Keyboard,2200.0,15
3,HD Webcam,1800.0,30
5,Gaming Headset,2800.0,18
6,Bluetooth Speaker,1499.0,10
```